

Beyond Softmax: A Natural Parameterization for Categorical Random Variables

Alessandro Manenti¹ Cesare Alippi^{1,2}

Abstract

Latent categorical variables are frequently found in deep learning architectures. They can model actions in discrete reinforcement-learning environments, represent categories in latent-variable models, or express relations in graph neural networks. Despite their widespread use, their discrete nature poses significant challenges to gradient-descent learning algorithms. While a substantial body of work has offered improved gradient estimation to improve training, we take a complementary approach. Specifically, we: 1) revisit the ubiquitous *softmax* function and demonstrate its limitations from an information-geometric perspective; 2) replace the *softmax* with the *catnat* function, a function composed of a sequence of hierarchical binary splits; we prove that this choice offers significant advantages to gradient descent due to the resulting diagonal Fisher Information Matrix. A rich set of experiments — including graph structure learning, variational autoencoders, and reinforcement learning — shows that the proposed function improves learning, yielding models with consistently higher test performance. *Catnat* is simple to implement and seamlessly integrates into existing codebases¹. Moreover, it remains compatible with standard training stabilization techniques and, as such, offers a better alternative to the *softmax* function.

1. Introduction

Categorical random variables — random variables that take one of a fixed set of values — are ubiquitous in machine

¹Università della Svizzera italiana, IDSIA, Lugano, Switzerland. ²Politecnico di Milano, Milan, Italy. Correspondence to: Alessandro Manenti <alessandro.manenti@usi.ch>.

Proceedings of the 43rd International Conference on Machine Learning, Seoul, South Korea. PMLR 306, 2026. Copyright 2026 by the author(s).

¹Implementations of *catnat* in PyTorch and JAX are available at www.github.com/allemamenti/catnat-torch and www.github.com/allemamenti/catnat-jax.

learning. They are used to represent a wide range of concepts, including classes in a classification problem (LeCun & Cortes, 2010), topics in a latent variable model (Miao et al., 2017), discrete actions in a reinforcement learning environment (Mnih et al., 2013), the presence or absence of connections in a graph (Franceschi et al., 2019) and clusters in mixture models (Jacobs et al., 1991).

The use of samples from categorical variables may be a modeling necessity or a choice dictated by scalability and efficiency. For instance, some problems are inherently discrete, such as selecting a word token in a language model (Chen et al., 2018; Paulus et al., 2020) or choosing an action in a reinforcement learning task (Mnih et al., 2013). In other cases, discretization is used for practical reasons, such as scalability in sparse graph modeling (Cini et al., 2023) or information compression using vector quantization in generative models (Van Den Oord et al., 2017).

In many of these settings, the categorical variables are latent, lacking a direct supervisory signal for training. The training signal must therefore be derived from an auxiliary loss function on a downstream task. While low-variance unbiased gradient estimators can be constructed for some continuous latent variables using techniques like the pathwise gradient estimator (Pflug, 1996; Kingma & Welling, 2014), the same methods are often not applicable to the discrete case. Consequently, learning often suffers from high-variance or biased

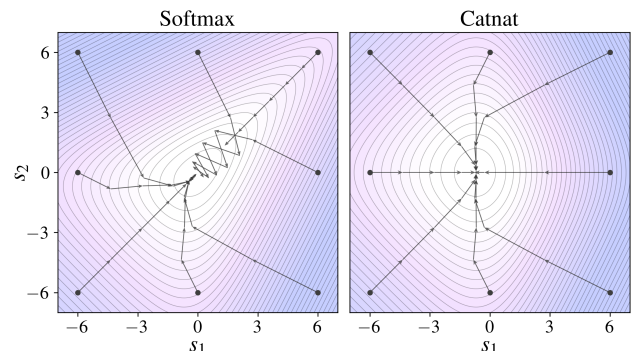


Figure 1. Cross-entropy loss landscapes for *softmax* and *catnat* parameterizations in a three-class classification problem with a uniform target distribution. The *catnat* landscape is smoother and more regular, yielding more direct gradient-descent trajectories (black lines). For *softmax*, the plot shows only a two-dimensional slice of its three-parameter landscape.

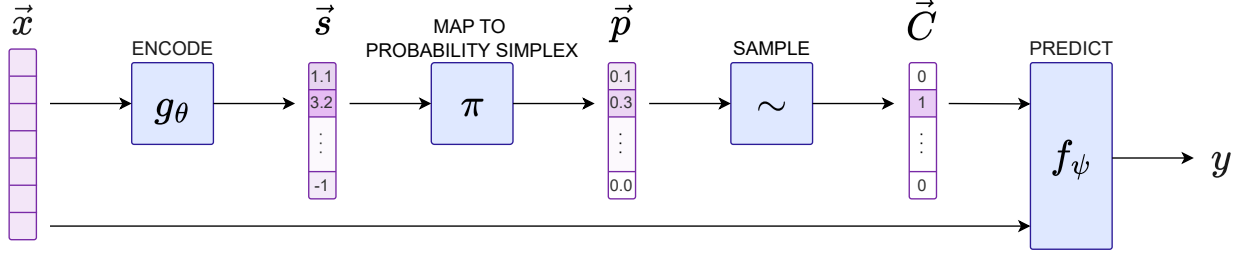


Figure 2. Schematic depiction of a model with a single categorical latent random variable. For rigor, we show $\vec{C}$ as a one-hot vector; however, in some cases (e.g., when using the standard version of the Gumbel–Softmax trick) $\vec{C}$ may be a dense vector.

gradient estimates, which can lead to unstable training runs that fail to converge to a satisfactory solution (Peters & Schaal, 2006). As a result, improving the training stability of models with latent categorical random variables remains an active area of research with potential for broad impact (Mohamed et al., 2020; Huijben et al., 2022; Ahmed et al., 2023).

Most techniques developed to stabilize the training of latent categorical distributions focus on reducing the variance of the gradient estimator. This is typically achieved by introducing novel control variates (Gu et al., 2016; Tucker et al., 2017), employing different sampling strategies (Kool et al., 2020), or designing new gradient estimators (Niepert et al., 2021). In this work we explore a complementary perspective: *we improve training effectiveness by changing the function that parameterizes the categorical distributions* within an information geometry-based framework. The modification we propose is simple to implement, can be easily integrated into existing codebases and is compatible with other training stabilization techniques.

To the best of our knowledge, this is the first work to use results from information geometry (Rao et al., 1945; Amari, 1998) to study the *softmax* parameterization and replace it with a function with better theoretical properties. Specifically, we observe that the standard *softmax* function has a dense Fisher Information Matrix (FIM), which induces geometric distortions in the parameter space. We therefore propose replacing it with a function designed to produce an optimization landscape more amenable to gradient-descent-based algorithms. This new parameterization – a series of hierarchical binary decisions that we call *catnat* – yields a diagonal FIM. This diagonal structure substantially reduces geometric distortions, allowing the optimizer to follow a more direct and stable path to a solution.

Figure 1 provides visual intuition in a simplified setting for the effect of a diagonal FIM on the optimization landscape. The diagonal FIM induced by *catnat* yields a more regular landscape than *softmax*, resulting in more direct gradient-descent trajectories, whereas *softmax* produces more oscillatory behavior. See Appendix A for a discussion of the

optimization benefits of a diagonal FIM.

Through extensive experiments in diverse settings – Graph Structure Learning (GSL), Variational Autoencoders (VAEs), and Reinforcement Learning (RL) – we empirically show that the proposed modification enables models to converge to solutions with superior final performance.

Implementations of *catnat* in PyTorch and JAX are available at www.github.com/allemantenti/catnat-torch and www.github.com/allemantenti/catnat-jax, respectively.

2. Problem Formulation

We consider models that employ a set of latent categorical variables to solve a downstream task. A pipeline general enough to include many deep learning models can be described as follows. (a) Given an input $x \in \mathcal{X}$, a neural network g_θ maps x to a vector of unnormalized scores $\vec{s} \in \mathbb{R}^S$. (b) These scores are transformed by a function $\pi : \mathbb{R}^S \rightarrow \Delta^{K-1}$ into a valid categorical probability vector $\vec{p}$ lying in the $(K-1)$ -dimensional probability simplex $\Delta^{K-1} := \{\vec{p} \in \mathbb{R}_{\geq 0}^K : \sum_{k=1}^K p_k = 1\}$. (c) A latent categorical variable $\vec{C}$ is then sampled according to $\text{Cat}(p_1, \dots, p_K)$ and (d) used, together with x , by a task-specific predictor f_ψ to produce the output y :

$$\begin{aligned} (a) \quad & \vec{s} = g_\theta(x), \\ (b) \quad & \vec{p} = \pi(\vec{s}) \quad \text{with} \quad \vec{p} = [p_1, \dots, p_K], \\ (c) \quad & \vec{C} \sim \text{Cat}(p_1, \dots, p_K), \\ (d) \quad & y = f_\psi(x, \vec{C}), \end{aligned} \tag{1}$$

A training signal is derived from y using a task-dependent objective (e.g., a supervised loss or a reinforcement-learning reward), and the model parameters (θ, ψ) are learned by gradient-based optimization of the corresponding expected objective. We present the overall architecture in Figure 2, with results naturally extending to more sophisticated architectural variants.

Depending on the application, g_θ , f_ψ , or both may be simple

neural networks as in VAEs, or compositions of parametric and non-parametric components as in RL. While we focus on a single categorical latent variable for clarity, the formulation and all subsequent results extend directly to collections of categorical variables with potentially different cardinalities.

3. Related Works and Preliminaries

Learning Categorical Variables The most common unbiased gradient estimator is the Score Function, or REINFORCE, gradient estimator (Williams, 1992). While unbiased, it suffers from high variance. This variance can be reduced using control variates (Ross, 2006) by subtracting a baseline from the learning signal. Simple baselines can be constructed by sampling the random variable multiple times, at the cost of introducing non-negligible computational overhead, or they can be estimated as a moving average from previous computations (Kool et al., 2019). More advanced control variates can be built efficiently using a neural network (Mnih & Gregor, 2014; Grathwohl et al., 2018), by employing a Taylor expansion of the mean-field network’s loss function (Gu et al., 2016), or by using a low-variance biased estimate of the loss (Tucker et al., 2017). Other gradient estimators can reduce variance at the expense of a biased gradient estimate by using a continuous relaxation of one-hot vectors, as in the Gumbel-Softmax (Jang et al., 2017; Maddison et al., 2017; Huijben et al., 2022), or by directly using mean-field gradients as a surrogate (Bengio et al., 2013). In the same class of biased estimators are MAP-based estimators (Niepert et al., 2021; Minervini et al., 2023) that derive a gradient signal from the change in the MAP estimate in response to perturbations of the distribution’s parameters.

Instead of changing the gradient estimator, another interesting line of research focuses on sampling techniques to reduce the variance of the estimator (Titsias & Lázaro-Gredilla, 2015). For example, Liu et al. (2019) propose to exactly compute the contribution of high probability components and to estimate the rest with an unbiased estimator while Kool et al. (2020) propose to sample without replacement and then unbiased the estimate to avoid duplicate samples. Often these techniques can often be interpreted as Rao-Blackwellizations (Mood et al., 1974) of simpler estimators. We remark that all the aforementioned techniques are orthogonal to our work and can be readily combined with it.

Information Geometry & Natural Gradient Ordinary gradient descent assumes an Euclidean geometry of the parameter space. In Amari (1998) and Amari & Douglas (1998) the authors recognized that the parameter space of

many learning models is not Euclidean and equal changes² in the parameter space can have disproportionate impacts on the model’s output distribution. To address this, they proposed measuring the ‘distance’ between parameter settings through the dissimilarity of their induced distributions, assessed by their Kullback–Leibler divergence. For distributions $p(x|\theta)$ and $p(x|\theta + d\theta)$ close in the parameter space, the KL divergence can be approximated as:

$$D_{KL}(p(x|\theta)||p(x|\theta + d\theta)) \simeq \frac{1}{2}d\theta^T G(\theta)d\theta \quad (2)$$

where $G(\theta)$ is the Fisher Information Matrix (FIM):

$$G(\theta) = \mathbb{E}_{p(x|\theta)} [(\nabla_{\theta} \log p(x|\theta)) (\nabla_{\theta} \log p(x|\theta))^T] \quad (3)$$

The FIM captures the local curvature of the statistical manifold (Amari, 2016) and the natural gradient is defined as the direction of steepest descent in this Riemannian manifold. The natural gradient $\tilde{\nabla} \mathcal{L}(\theta)$ is obtained by pre-conditioning the ordinary gradient with the inverse of the FIM:

$$\tilde{\nabla} \mathcal{L}(\theta) = G(\theta)^{-1} \nabla \mathcal{L}(\theta) \quad (4)$$

While theoretically advantageous, implementing natural gradient descent presents practical challenges. First, for each update step, computing Equation (4) requires calculating and inverting the FIM, which entails cubic scaling and can easily become a computational bottleneck. For this reason, different approximations have been proposed (Pascanu & Bengio, 2013; Grosse & Martens, 2016; Amari et al., 2019) to speed up computation at the expense of precision. As a second problem, Equations (2) and (4) hold true for infinitesimal parameter changes, so their approximation error increases with larger, more practical step sizes commonly used during optimization. In Martens (2020) natural gradient descent is analyzed from the perspective of a second-order optimization method, demonstrating that designing a robust natural gradient optimizer necessitates the incorporation of techniques such as trust regions and Tikhonov regularization. Furthermore, the FIM can be singular or create numerical instabilities during its inversion. In this work, we propose to tackle both problems by choosing a suitable parameterization for the categorical latent random variable that intrinsically produces a diagonal FIM. See Appendix A for further intuition on why a diagonal Fisher Information Matrix is desirable.

Hierarchical Classifiers and Catnat Hierarchical decompositions of categorical predictions have been used in multiclass classification (Morin & Bengio, 2005; Mnih & Hinton, 2008; Beygelzimer et al., 2009) to reduce prediction or probability-estimation cost to $\mathcal{O}(\log(K))$. In contrast, *catnat* arises from information-geometric considerations,

²measured by some kind of Euclidean norm.

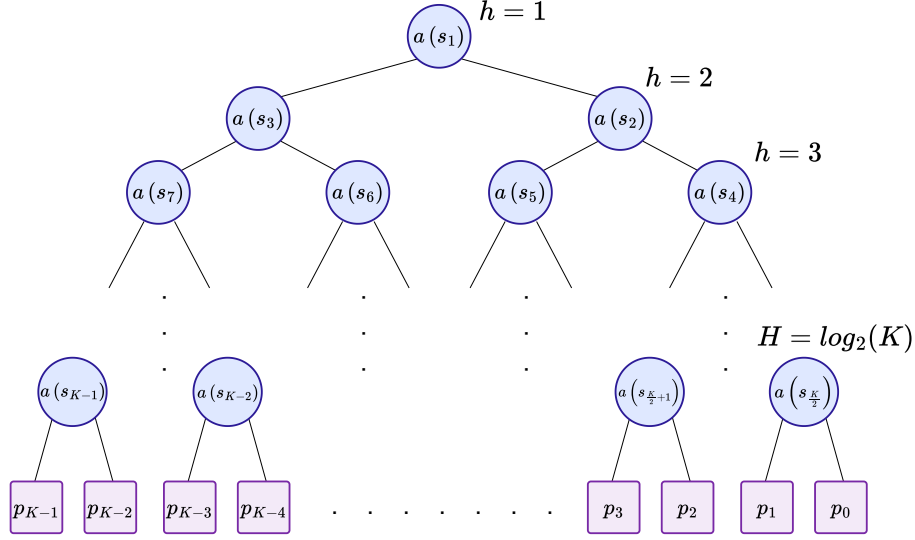


Figure 3. *Catnat* parameterization for the categorical distribution. Given unnormalized scores s_i and activation function a , blue nodes compute the probability of going left ($a(s_i)$) or right ($1 - a(s_i)$). Final categorical probabilities are shown in purple. On the right side, the hierarchy level h is indicated. Note that indices for p start from zero, while indices for s start from one.

yielding a broader class of categorical parameterizations with favorable optimization properties. Under this view, some existing hierarchical classifiers are special cases of *catnat*, obtained by choosing particular binary activation functions $a(\cdot)$ at the internal nodes.

4. Categorical Random Variables Parameterizations

4.1. The Softmax Function and its Pitfalls

Let $\vec{s} \in \mathbb{R}^{1 \times K}$ be a set of scores. The *softmax* function is defined as:

$$p_i = \frac{e^{s_i}}{\sum_{k=1}^K e^{s_k}} \quad (5)$$

Originally introduced in statistical mechanics as the Boltzmann distribution (Jaynes, 1957), the *softmax* later appeared in statistics as the canonical link for categorical outcomes in Generalized Linear Models (GLM) (Nelder & Wedderburn, 1972). In neural networks, the name *softmax* was popularized by Bridle (1989), who also describes some of its appealing properties: it converts arbitrary real vectors into non-negative probabilities, preserves rank order, and offers a smooth approximation to the $\arg \max$. These features have made it the standard parameterization for categorical variables in machine learning.

Despite its benefits, the *softmax* function has non-negligible drawbacks. It is overparameterized, using K parameters to represent a $(K - 1)$ -dimensional simplex, it can saturate, leading to vanishing gradients (Goodfellow et al., 2016) and, in highly nonlinear probabilistic models, the GLM

assumptions behind its usefulness (Nelder & Wedderburn, 1972) may not hold.

We argue that Information Geometry (Amari, 1998; 2016) provides a principled framework for defining more suitable parameterizations. To this end, in Proposition 4.1 we analyze the geometric properties induced by the *softmax* function.

Proposition 4.1. *The Fisher Information Matrix for a categorical random variable parameterized by the softmax function, as defined in (5), is*

$$G_{\text{smx}}(s) = \begin{bmatrix} p_1(1 - p_1) & -p_1p_2 & \cdots & -p_1p_K \\ -p_2p_1 & p_2(1 - p_2) & \cdots & -p_2p_K \\ \vdots & \vdots & \ddots & \vdots \\ -p_Kp_1 & -p_Kp_2 & \cdots & p_K(1 - p_K) \end{bmatrix} \quad (6)$$

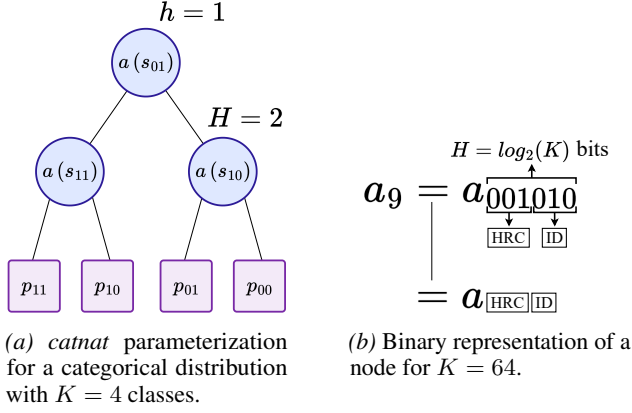
We provide a proof of the proposition in Appendix B.

The resulting Fisher Information Matrix is dense, with off-diagonal entries $-p_i p_j$ that couple all scores. Consequently, the statistical manifold is curved, and, as discussed in Section 3 and by Amari & Douglas (1998); Amari (2016), gradient-based optimization becomes less accurate. To address this, we introduce a class of parameterizations designed to induce a flatter statistical manifold.

4.2. Catnat: A Class of Natural Parameterizations

4.2.1. CATNAT DEFINITION

In this section, we propose a class of parameterizations for categorical random variables that, as demonstrated in


 Figure 4. Simple examples for the *catnat* parameterization.

Theorem 4.2, yield a diagonal Fisher Information Matrix. We refer to this class as *catnat*, as it parametrizes the categorical distribution in accordance with natural gradient principles. By analyzing the general form of the FIM, we further identify and select the parameterization with the minimal number of factors in the diagonal terms.

The proposed class models the categorical probability distribution as the outcome of a sequence of binary decisions, structured as a hierarchical tree. Each unnormalized score s_i corresponds to a unique node in this tree. To ensure that the resulting probabilities lie in the interval $[0, 1]$, an activation function $a : \mathbb{R} \rightarrow [0, 1]$ is applied to each score. Figure 3 illustrates this construction.

Each score s_i , its corresponding binary probability $a_i := a(s_i)$, and the final categorical probabilities p_k are associated with unique indices. To ease the theoretical analysis, we express those indices in their binary representation that, for a categorical distribution over K classes, can be represented by a string of length $H = \log_2(K)$ bits. See Figure 4a for an example with $K = 4$.

For Bernoulli probabilities a_i and scores s_i , we introduce a convenient notation by splitting the binary string into two sequences: $\overline{\text{HRC}}$ and $\overline{\text{ID}}$. Given the hierarchy level h of a_i in the tree, $\overline{\text{HRC}}$ consists of $h - 1$ zeros followed by a one, while $\overline{\text{ID}}$ specifies the position of the node at that level. For example, in Figure 4b, $\overline{\text{HRC}}$ shows that a_9 is at hierarchy level $h = 3$, and $\overline{\text{ID}} = 010$ identifies it as the second node from the right.

Since $\overline{\text{HRC}}$ is uniquely determined once $\overline{\text{ID}}$ and K are given, we drop $\overline{\text{HRC}}$ and write, for a node at hierarchy h :

$$a_{\overline{\text{HRC}}\overline{\text{ID}}} = a_{\overline{\text{ID}}} = a_{[b_1, \dots, b_{h-1}]} \quad (7)$$

The introduced notation allows for a compact representation of the categorical probabilities. Specifically, the probability p_k of category k identified by the binary string $\vec{b} = [b_1, \dots, b_H]$ is:

$$p_{\vec{b}} = p_{b_1, \dots, b_H} = \prod_{h=1}^H \left(a_{[b_1, \dots, b_{h-1}]} \right)^{b_h} \left(1 - a_{[b_1, \dots, b_{h-1}]} \right)^{1-b_h} \quad (8)$$

At the root node ($h = 1$), the path represented by the set $b_1, \dots, b_{h-1}$ is empty, consistent with the fact that the node is uniquely identified by its hierarchy level alone. By construction, the probability of descending from the root to a node a_i is:

$$\begin{aligned} P(a_i) &= P(a_{\overline{\text{ID}}_i}) = P(a_{[b_1, \dots, b_{h-1}]}) = \\ &= \prod_{h=1}^{h_i-1} \left(a_{[b_1, \dots, b_{h-1}]} \right)^{b_h} \left(1 - a_{[b_1, \dots, b_{h-1}]} \right)^{1-b_h} \end{aligned} \quad (9)$$

This probability is also equivalent to the sum of the probabilities of all leaf nodes $p_{\vec{b}}$ that descend from the node a_i :

$$P(a_i) = \sum_{\vec{b} \in \mathcal{D}_i} p_{\vec{b}} \quad (10)$$

where $\mathcal{D}_i$ is the set of leaf nodes in the subtree rooted at a_i .

4.2.2. THEORETICAL ADVANTAGES OF CATNAT

The following theorem shows that the *catnat* parameterization is better suited for gradient-based optimization than the *softmax*.

Theorem 4.2. *The Fisher Information Matrix $G_a(s)$ for the *catnat* parameterization is:*

$$G_a(s)_{ij} = \begin{cases} 0 & \text{if } i \neq j \\ P(a_i) \left(\frac{\partial a_i}{\partial s_i} \right)^2 \left(\frac{1}{a_i(1-a_i)} \right) & \text{if } i = j \end{cases} \quad (11)$$

The proof is provided in Appendix C. Theorem 4.2 shows that this class of parameterizations yields diagonal FIMs, thereby flattening the optimization manifold. The diagonal entries, $G_a(s)_{ii}$, depend on two components: the probability of reaching node i , $P(a_i)$, and a term involving the derivative of the chosen activation function. Since $P(a_i)$ is determined by the scores of all ancestor nodes, each diagonal entry depends on at most $H = \log_2(K)$ scores. Furthermore, as the overall complexity of the FIM is governed by the choice of $a(s)$, we can further simplify Equation (11). We propose the *natural* activation function $\nu(x)$ to render the second component constant:

$$\nu(x) = \begin{cases} 0 & \text{if } x \leq C - \frac{A}{2} \\ \frac{1 + \sin\left(\frac{\pi(x-C)}{A}\right)}{2} & \text{if } C - \frac{A}{2} \leq x \leq C + \frac{A}{2} \\ 1 & \text{if } x \geq C + \frac{A}{2} \end{cases} \quad (12)$$

C is a parameter that can be used to shift the function along the x axis and to modify the categorical probabilities at

initializations – when it is reasonable to expect the scores to be distributed around zero. In the experiments we use $C = 0$. Parameter A can be changed to modify the slope of the function around C . In the experiments, to make a fair comparison between the natural activation ν and the sigmoid function σ we set A so that $\frac{\partial \nu}{\partial s}|_{s=0} = \frac{\partial \sigma}{\partial s}|_{s=0}$ resulting in $A = 2\pi$. Note that neither A nor C are additional hyperparameters to be tuned.

We term ν the *natural* activation function as it simplifies the FIM in a way that aligns with the objectives of natural gradient methods, as demonstrated in Corollary 4.3.

Corollary 4.3. *The Fisher Information Matrix $G_a(s)$ for the catnat parameterization using the natural activation function ν is:*

$$G_\nu(s)_{ij} = \begin{cases} 0 & \text{if } i \neq j \\ P(a_i) \left(\frac{\pi}{A}\right)^2 & \text{if } i = j \text{ and } |s_i - C_i| < \frac{A}{2} \end{cases} \quad (13)$$

with the value for $i = j$ at $|s_i - C_i| = \frac{A}{2}$ defined by continuity.

The corollary is proved in Appendix D. The corollary shows that using the *natural* activation eliminates the dependence of each diagonal entry $G_\nu(s)_{ii}$ on the local score s_i , leaving only the ancestor-dependent probability term $P(a_i)$.

4.2.3. CATNAT SIMPLIFIES THE GLOBAL FIM

The diagonal FIM in Theorem 4.2 is local to the categorical parameterization: it is defined with respect to the intermediate scores $\vec{s}$, not with respect to the network parameters θ that are optimized. It therefore does not imply that the FIM with respect to θ is diagonal – and in general it is not. What it does guarantee is that all cross-score statistical coupling introduced by the categorical parameterization disappears. The following derivation makes this precise.

By the standard transformation law of the Fisher metric under reparameterization (Amari & Nagaoka, 2000; Amari, 2016), the FIM with respect to θ is obtained by pulling back the score-space FIM through the Jacobian of g_θ . Let $\vec{s} = g_\theta(x) \in \mathbb{R}^S$ and let $J_\theta(x) := \partial \vec{s} / \partial \theta$. For a categorical parameterization $\vec{p} = \pi(\vec{s})$, let $G_\pi(\vec{s})$ denote the FIM with respect to $\vec{s}$. Then, for a fixed input x ,

$$G_\theta(\theta; x) = J_\theta(x)^T G_\pi(\vec{s}) J_\theta(x) = \sum_{i,j} G_\pi(\vec{s})_{ij} \nabla_\theta s_i (\nabla_\theta s_j)^T \quad (14)$$

For *catnat*, Theorem 4.2 gives $G_\pi(\vec{s}) = G_a(\vec{s})$, where $G_a(\vec{s})$ is diagonal. The off-diagonal contributions in (14) therefore vanish, leaving

$$G_\theta^{\text{catnat}}(\theta; x) = \sum_{i=1}^S G_a(\vec{s})_{ii} \nabla_\theta s_i (\nabla_\theta s_i)^T. \quad (15)$$

Table 1. Different datasets are generated with different latent distributions. The latent distribution is determined by the Bernoulli probability θ^* of sampling edges from communities as in Fig. 5.

True Bernoulli probability θ^*	Binary entropy per edge (shannons)
0.1	0.47
0.25	0.81
0.5	1
0.75	0.81
0.9	0.47

This does not make G_θ^{catnat} diagonal: the Jacobian $J_\theta(x)$ can still couple network parameters. It does, however, remove every cross-score term induced by the categorical parameterization. By contrast, the *softmax* FIM is dense, with off-diagonal entries $-p_i p_j$, so its pulled-back FIM contains explicit cross-score interactions $\nabla_\theta s_i (\nabla_\theta s_j)^T$ for $i \neq j$.

5. Experiments

We evaluate four parameterizations for categorical latent random variables: the *softmax* function, the *sparsemax* function (Martins & Astudillo, 2016), the *catnat* parameterization with *sigmoid* activation, and the *catnat* parameterization with *natural* activation function. The evaluation spans three distinct domains that rely on such variables: Graph Structure Learning (GSL), Variational Autoencoders (VAE), and Reinforcement Learning (RL). These domains allow us to assess the proposed method under diverse conditions, varying factors such as the gradient estimator employed for the latent distribution parameters, the number of categories (K), the number of latent variables (N), the form of the loss or reward function and the downstream task considered. Empirical results show that both hierarchical parameterizations typically converge to better optima, with the proposed *natural* activation function yielding superior performance in the majority of the cases.

5.1. Graph Structure Learning

Graph Neural Networks (GNNs) (Scarselli et al., 2008) are a class of models that leverage relational information, encoded in an adjacency matrix A , as an inductive bias to improve performance on various predictive tasks (Fout et al., 2017; Shlomi et al., 2020). Often, the optimal graph structure is not available and must be inferred from the data, a process known as Graph Structure Learning (GSL) (Kipf et al., 2018; Franceschi et al., 2019; Fatemi et al., 2021). In this context, the adjacency matrix A is frequently treated as a collection of latent categorical random variables $\vec{C}$, where a Bernoulli random variable typically models the existence of each edge (Franceschi et al., 2019; Elinas et al., 2020; Zambon et al., 2023; Cini et al., 2023; Manenti et al., 2025).

We adopt the experimental setup from Manenti et al. (2025),

Table 2. Test metrics of models trained on datasets generated with different true latent parameters θ^* . ES, PP-MAE, and PP-MSE measure predictive performance, while MAE in θ evaluates calibration on the latent distribution parameters. For all metrics, lower values indicate better performance. **Bold** numbers indicate the best-performing models (p-value of the Welch’s t-test < 0.05). Optimal values are estimated using the true generating model.

θ^*	Activation	ES loss	PP-MAE	PP-MSE	MAE on θ
0.1	<i>sparsemax</i>	7.467 $\pm$ 0.013	0.3842 $\pm$ 0.0012	0.624 $\pm$ 0.002	0.0083 $\pm$ 0.0012
	<i>softmax</i>	7.450 $\pm$ 0.015	0.3844 $\pm$ 0.0012	0.622 $\pm$ 0.003	0.0077 $\pm$ 0.0003
	<i>sigmoid</i>	7.459 $\pm$ 0.018	0.3845 $\pm$ 0.0010	0.623 $\pm$ 0.003	0.0090 $\pm$ 0.0003
	<i>natural</i>	7.425 $\pm$ 0.014	0.3837 $\pm$ 0.0015	0.617 $\pm$ 0.004	0.0052 $\pm$ 0.0003
	Optimal value	7.416 $\pm$ 0.014	0.3849 $\pm$ 0.0009	0.615 $\pm$ 0.002	0
0.25	<i>sparsemax</i>	10.928 $\pm$ 0.023	0.8224 $\pm$ 0.0007	1.321 $\pm$ 0.007	0.0123 $\pm$ 0.0011
	<i>softmax</i>	10.882 $\pm$ 0.019	0.8224 $\pm$ 0.0013	1.312 $\pm$ 0.004	0.0083 $\pm$ 0.0002
	<i>sigmoid</i>	10.922 $\pm$ 0.010	0.8230 $\pm$ 0.0014	1.319 $\pm$ 0.003	0.0086 $\pm$ 0.0005
	<i>natural</i>	10.859 $\pm$ 0.012	0.8201 $\pm$ 0.0012	1.304 $\pm$ 0.003	0.0051 $\pm$ 0.0003
	Optimal value	10.842 $\pm$ 0.014	0.8192 $\pm$ 0.0012	1.300 $\pm$ 0.003	0
0.5	<i>sparsemax</i>	15.030 $\pm$ 0.030	1.2592 $\pm$ 0.0042	2.510 $\pm$ 0.020	0.0126 $\pm$ 0.0011
	<i>softmax</i>	14.990 $\pm$ 0.020	1.2596 $\pm$ 0.0015	2.485 $\pm$ 0.007	0.0132 $\pm$ 0.0008
	<i>sigmoid</i>	15.020 $\pm$ 0.015	1.2607 $\pm$ 0.0016	2.490 $\pm$ 0.007	0.0101 $\pm$ 0.0005
	<i>natural</i>	14.937 $\pm$ 0.023	1.2537 $\pm$ 0.0019	2.466 $\pm$ 0.007	0.0061 $\pm$ 0.0006
	Optimal value	14.926 $\pm$ 0.018	1.2523 $\pm$ 0.0019	2.462 $\pm$ 0.005	0
0.75	<i>sparsemax</i>	10.979 $\pm$ 0.102	0.8278 $\pm$ 0.0188	1.343 $\pm$ 0.047	0.0137 $\pm$ 0.0022
	<i>softmax</i>	10.713 $\pm$ 0.096	0.8044 $\pm$ 0.0027	1.278 $\pm$ 0.002	0.0146 $\pm$ 0.0006
	<i>sigmoid</i>	10.750 $\pm$ 0.019	0.8078 $\pm$ 0.0020	1.280 $\pm$ 0.004	0.0119 $\pm$ 0.0004
	<i>natural</i>	10.674 $\pm$ 0.012	0.7969 $\pm$ 0.0015	1.267 $\pm$ 0.004	0.0043 $\pm$ 0.0002
	Optimal value	10.672 $\pm$ 0.017	0.7943 $\pm$ 0.0013	1.267 $\pm$ 0.003	0
0.9	<i>sparsemax</i>	7.592 $\pm$ 0.136	0.4399 $\pm$ 0.0196	0.651 $\pm$ 0.044	0.0102 $\pm$ 0.0019
	<i>softmax</i>	7.374 $\pm$ 0.012	0.4272 $\pm$ 0.0013	0.614 $\pm$ 0.002	0.0108 $\pm$ 0.0003
	<i>sigmoid</i>	7.398 $\pm$ 0.017	0.4228 $\pm$ 0.0018	0.616 $\pm$ 0.003	0.0085 $\pm$ 0.0003
	<i>natural</i>	7.340 $\pm$ 0.015	0.3973 $\pm$ 0.0015	0.607 $\pm$ 0.003	0.0023 $\pm$ 0.0001
	Optimal value	7.342 $\pm$ 0.016	0.3839 $\pm$ 0.0014	0.611 $\pm$ 0.002	0

generating synthetic data with a Graph Neural Network (GNN), $f_{\psi^*}(x, A)$. This GNN computes an output y^* from random input features x and a latent graph A , which we sample from a multivariate Bernoulli distribution, $P_{\theta^*}(A)$. The ground-truth parameters θ_{ij}^* are set to the same non-zero value θ^* for edges forming the community structure depicted in Figure 5 and are zero otherwise. We use this dataset to train a model with an identical architecture to recover the underlying graph structure and GNN parameters. We optimize the model using the Energy Score (ES) (Gneiting & Raftery, 2007) loss for its calibration advantages (Manenti et al., 2025). We use the score function gradient estimator (Williams, 1992) with the LOO baseline to train the latent parameters. We provide additional details in Appendix E.

To compare the score parameterizations under different entropy settings we generate five datasets with different true latent parameters θ^* . Experiment configurations are detailed in Table 1. The task in this setting is twofold: (i) to make optimal point predictions, measured for example by the Point Prediction Mean Absolute Error (PP-MAE)

and Mean Squared Error (PP-MSE), and (ii) to learn the correct graph structure, i.e., to accurately estimate the true parameters θ^* . The latter is evaluated, for example, by the mean absolute error on the distribution parameters (MAE on θ), $\langle |\theta_{ij} - \theta_{ij}^*| \rangle$.

The experimental results in Table 2 show that the *natural* activation ν consistently outperforms the *sparsemax*, *sigmoid* and *softmax* parameterization across all metrics and data-generating conditions, with the largest gains in learning the underlying latent distribution. In particular, the natural parameterization recovers the true data-generating parameters more accurately, as measured by the MAE on θ . In terms of predictive performance, the natural parameterization also achieves lower mean scores on ES and both point prediction errors.

5.2. Categorical VAE

Variational autoencoders (VAEs) (Kingma & Welling, 2014) constitute a class of deep generative models that learn compact latent representations of data via a probabilistic framework. The original VAE framework employs a continuous

Table 3. Test set negative log likelihood on the MNIST dataset. Negative log-likelihoods are estimated with 512 importance samples (Burda et al., 2016). Models are compared across the number of categorical variables N , categories K , and categorical parameterizations. **Bold** denotes the best-performing models (p-value of the Welch’s t-test < 0.05) for each (N, K) setting. Adam used for optimization.

N	Param.	MNIST			Binary MNIST		
		$K = 8$	$K = 16$	$K = 32$	$K = 8$	$K = 16$	$K = 32$
10	<i>sparsemax</i>	102.5 $\pm$ 0.3	101.1 $\pm$ 0.6	101.3 $\pm$ 0.8	87.0 $\pm$ 0.4	83.4 $\pm$ 0.9	85.2 $\pm$ 1.5
	<i>softmax</i>	100.9 $\pm$ 0.5	98.1 $\pm$ 0.7	98.6 $\pm$ 0.7	84.9 $\pm$ 0.8	81.0 $\pm$ 1.2	79.9 $\pm$ 0.5
	<i>catnat</i> σ	99.5 $\pm$ 0.2	97.7 $\pm$ 0.4	96.6 $\pm$ 0.2	83.0 $\pm$ 0.6	78.8 $\pm$ 0.6	76.9 $\pm$ 0.7
	<i>catnat</i> ν	99.8 $\pm$ 0.4	97.6 $\pm$ 0.2	96.9 $\pm$ 0.4	83.2 $\pm$ 0.5	78.7 $\pm$ 0.3	77.3 $\pm$ 0.4
20	<i>sparsemax</i>	102.1 $\pm$ 0.7	102.1 $\pm$ 0.8	103.8 $\pm$ 0.7	84.3 $\pm$ 0.8	83.6 $\pm$ 0.8	85.1 $\pm$ 0.6
	<i>softmax</i>	97.8 $\pm$ 0.2	97.5 $\pm$ 0.5	98.2 $\pm$ 0.8	78.3 $\pm$ 0.5	78.1 $\pm$ 0.4	79.2 $\pm$ 1.0
	<i>catnat</i> σ	97.5 $\pm$ 0.3	96.9 $\pm$ 0.4	97.0 $\pm$ 0.3	77.5 $\pm$ 1.1	76.7 $\pm$ 0.7	76.2 $\pm$ 0.5
	<i>catnat</i> ν	97.7 $\pm$ 0.2	97.0 $\pm$ 0.4	96.9 $\pm$ 0.4	77.1 $\pm$ 0.4	76.6 $\pm$ 0.3	76.8 $\pm$ 0.4
30	<i>sparsemax</i>	105.3 $\pm$ 0.5	103.9 $\pm$ 0.5	104.9 $\pm$ 0.8	86.9 $\pm$ 0.6	87.2 $\pm$ 1.0	88.7 $\pm$ 1.3
	<i>softmax</i>	98.8 $\pm$ 0.7	98.8 $\pm$ 0.9	99.3 $\pm$ 0.7	79.0 $\pm$ 0.5	79.2 $\pm$ 0.9	80.6 $\pm$ 0.6
	<i>catnat</i> σ	98.1 $\pm$ 0.4	97.6 $\pm$ 0.4	97.9 $\pm$ 0.5	77.9 $\pm$ 0.7	77.8 $\pm$ 0.6	77.9 $\pm$ 1.1
	<i>catnat</i> ν	97.9 $\pm$ 0.3	97.6 $\pm$ 0.3	97.7 $\pm$ 0.8	77.9 $\pm$ 0.6	77.7 $\pm$ 0.5	78.0 $\pm$ 0.7

latent distribution, typically Gaussian, which may not suit data with inherently discrete factors (Van Den Oord et al., 2017). To address this limitation, VAEs have been extended to incorporate categorical variables $\vec{C}$, enabling more accurate modeling of such structures (Jang et al., 2017; Maddison et al., 2017). In this configuration, the latent space is parameterized by one or more categorical random variables.

We trained a variational autoencoder (VAE) with a discrete, categorical latent space on the MNIST dataset (LeCun & Cortes, 2010) and on a binarized version of the same dataset, where pixels are thresholded at 0.5 of their maximum intensity value (Akrami et al., 2022). Its encoder, a Convolutional Neural Network, processes an input image x to produce a tensor of unnormalized scores, $\vec{s} \in \mathbb{R}^{N \times K}$, which parameterize the approximate posterior distribution $q(\vec{C}|x)$. The latent space is structured as a composite of N independent categorical variables, where each variable can assume one of K distinct classes. To enable gradient flow through the discrete sampling process, we employ the Gumbel-Softmax trick (Jang et al., 2017), which generates differentiable sample tensors $\vec{C} \in [0, 1]^{N \times K}$. Note that the samples used by the Gumbel-Softmax trick are continuous relaxations of one-hot vectors. The latent samples $\vec{C}$ are then fed into the decoder, a corresponding transposed convolutional network (Zeiler et al., 2010), to generate a reconstructed image $\hat{y}$.

To form the categorical distributions from the encoder’s scores, we test four parameterization schemes: the *softmax* function, the *sparsemax* function, the *catnat* function with *sigmoid* activation function and the *catnat* function with *natural* activation. The model’s training objective is the minimization of the Evidence Lower Bound (ELBO) (Kingma & Welling, 2014). We defer additional experimental details to Appendix F.

The results in Table 3 show a clear performance advantage for both *catnat* parameterizations over both the *softmax* and the *sparsemax* across all experiments. This indicates that the benefits of the proposed parameterization are relevant in the probabilistic generative modeling case. Importantly, these improvements are observed across a wide range of latent configurations ($N \in \{10, 20, 30\}$, $K \in \{8, 16, 32\}$), underscoring the robustness of the approach to changes in model capacity and latent space complexity. Within the hierarchical methods, the natural activation function ν yields a slight improvement over the sigmoid function σ in the majority of the settings, although the two are statistically equivalent overall.

Since the proposed parameterization is grounded in Information Geometry and directly targets issues arising in gradient-based optimization, we run the same set of experiments using SGD instead of Adam as the optimizer. The results in Table 5 (Appendix G) confirm that the performance patterns observed with Adam extend to other optimizers, with the *catnat* parameterizations consistently outperforming the *sparsemax* and *softmax* baselines.

We ran an additional experiment using a different implementation of a discrete VAE, based on the codebase of Jeffares & Liu (2025). Even in this setting, the best-performing models use the proposed *catnat* parameterization and outperform the *softmax* baseline. Additional details are provided in Appendix H.

Overall, these results demonstrate that in practical scenarios, replacing the standard *softmax* with the proposed *catnat* parameterization facilitates optimization and improves downstream performance, in line with theoretical results.

Table 4. Final episodic returns on Seaquest and Breakout environments. The higher the better. **Bold** denotes the best-performing models (p-value of the Welch’s t-test < 0.05)

Parameterization	RL Environment	
	Breakout	Seaquest
<i>softmax</i>	398 $\pm$ 25	1875 $\pm$ 312
<i>catnat</i> ν	406 $\pm$ 34	2164 $\pm$ 533

5.3. Reinforcement Learning

Reinforcement learning (RL) is a framework where an agent learns to make sequential decisions by interacting with an environment in order to maximize a cumulative reward (Sutton et al., 1998). In policy-based approaches (Sutton et al., 1998), the agent’s strategy is directly parameterized by a policy π , which maps observed states to actions. In many domains, such as board games or Atari video games (Belle-mare et al., 2013; Mnih et al., 2013; 2015), the action space is discrete, requiring the agent to select from a finite set of choices at each step. In such settings, the policy produces a categorical distribution over the available actions.

In this setting, we employ the Proximal Policy Optimization (PPO) algorithm (Schulman et al., 2017) on the discrete-action Atari environments Breakout and Seaquest (Mnih et al., 2013; 2015). We adopt the PPO implementation of Huang et al. (2022) in which an agent uses a shared-parameter actor-critic architecture, with a deep convolutional network processing stacked game frames to produce a latent state representation. This state is then fed into two separate heads: a value head that estimates the state-value function, and a policy head that outputs scores for the action distribution. Additional experimental details are provided in Appendix I

Due to the computational burden of these experiments, an exhaustive hyperparameter search was not feasible. Instead, for each method and environment, we selected promising configurations by sampling 160 trials with a Tree-structured Parzen Estimator (TPE) Bayesian sampler (Bergstra et al., 2011). The top 10 resulting configurations were then re-evaluated across 10 independent random seeds to gather performance statistics. Within this framework, we tested two methods to convert the policy head’s scores into action probabilities: the standard *softmax* function and the *catnat* using the *natural* activation function.

Table 4 reports the final episodic returns on Seaquest and Breakout environments. The *catnat* parameterization yields better performance than the standard *softmax* function, with a modest improvement in Breakout and a more substantial gain in the more complex Seaquest environment. These results indicate that the information-geometric properties of *catnat* translate into practical benefits even in high-dimensional, sequential decision-making tasks. The fact

that *catnat* maintains a consistent performance advantage despite this variance suggests that its benefits are robust rather than artifacts of specific hyperparameter settings. In particular, the larger relative gains in Seaquest, where the action space is richer and exploration dynamics more complex, point to potential advantages in environments with increased complexity. A more exhaustive search could provide a clearer picture of the potential performance ceiling of *catnat*, with future work investigating how the relative benefits of this parameterization scale with task difficulty, action space size, or agent capacity.

6. Conclusions

We introduced a new perspective for improving training of models with latent categorical random variables. Specifically, we showed that replacing the standard *softmax* parameterization with a *catnat* function – a hierarchical sequence of binary decisions – yields favorable information-geometric properties. In accordance with a substantial body of literature (Amari, 1998; Amari & Douglas, 1998; Amari & Nagaoka, 2000; Martens, 2020), our empirical results across diverse settings indicate that better information-geometric properties facilitate gradient-based optimization. In particular, important performance improvements follow by simply replacing the *softmax* function with the *catnat*.

Two main directions for future work remain. First, although our study focused on categorical distributions, the findings suggest that parameterizations that induce a diagonal Fisher Information Matrix consistently improve performance. Extending this approach to other families of continuous and discrete distributions is a promising avenue for future research. Second, our experiments were not designed to engineer models for state-of-the-art performance, but rather to demonstrate the broad applicability and effectiveness of the proposed approach. Application-specific state-of-the-art methods that rely on categorical random variables are expected to benefit from the *catnat* parameterization, and can achieve new state-of-the-art results with minimal effort.

Impact Statement

This work presents fundamental research in machine learning with broad applicability across multiple areas, including generative models, reinforcement learning, and graph neural networks. While the potential impact is wide-ranging, we do not identify a specific direct societal consequence.

Acknowledgments

This work was supported by the Swiss National Science Foundation project FNS 204061: *HORD GNN: Higher-Order Relations and Dynamics in Graph Neural Networks*

References

- Ahmed, K., Zeng, Z., Niepert, M., and Van den Broeck, G. Simple: A gradient estimator for k-subset sampling. In *The Eleventh International Conference on Learning Representations*, 2023.
- Akrami, H., Joshi, A. A., Li, J., Aydoore, S., and Leahy, R. M. A robust variational autoencoder using beta divergence. *Knowledge-based systems*, 238:107886, 2022.
- Amari, S.-i. Natural gradient works efficiently in learning. *Neural computation*, 10(2):251–276, 1998.
- Amari, S.-i. *Information geometry and its applications*, volume 194. Springer, 2016.
- Amari, S.-i. and Douglas, S. C. Why natural gradient? In *Proceedings of the 1998 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP’98 (Cat. No. 98CH36181)*, volume 2, pp. 1213–1216. IEEE, 1998.
- Amari, S.-i. and Nagaoka, H. *Methods of information geometry*, volume 191. American Mathematical Soc., 2000.
- Amari, S.-i., Karakida, R., and Oizumi, M. Fisher information and natural gradient learning in random deep networks. In *The 22nd International Conference on Artificial Intelligence and Statistics*, pp. 694–702. PMLR, 2019.
- Bellemare, M. G., Naddaf, Y., Veness, J., and Bowling, M. The arcade learning environment: An evaluation platform for general agents. *Journal of artificial intelligence research*, 47:253–279, 2013.
- Bengio, Y., Léonard, N., and Courville, A. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.
- Bergstra, J., Bardenet, R., Bengio, Y., and Kégl, B. Algorithms for hyper-parameter optimization. *Advances in neural information processing systems*, 24, 2011.
- Beygelzimer, A., Langford, J., Lifshits, Y., Sorkin, G., and Strehl, A. Conditional probability tree estimation analysis and algorithms. In *Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence*, pp. 51–58, 2009.
- Bridle, J. Training stochastic model recognition algorithms as networks can lead to maximum mutual information estimation of parameters. *Advances in neural information processing systems*, 2, 1989.
- Burda, Y., Grosse, R., and Salakhutdinov, R. Importance weighted autoencoders. In *4th International Conference on Learning Representations (ICLR)*, 2016.
- Chen, J., Song, L., Wainwright, M., and Jordan, M. Learning to explain: An information-theoretic perspective on model interpretation. In *International conference on machine learning*, pp. 883–892. PMLR, 2018.
- Cini, A., Zambon, D., and Alippi, C. Sparse graph learning from spatiotemporal time series. *Journal of Machine Learning Research*, 24:1–36, 2023.
- Elinas, P., Bonilla, E. V., and Tiao, L. Variational inference for graph convolutional networks in the absence of graph data and adversarial settings. *Advances in Neural Information Processing Systems*, 33:18648–18660, 2020.
- Fatemi, B., El Asri, L., and Kazemi, S. M. Slaps: Self-supervision improves structure learning for graph neural networks. *Advances in Neural Information Processing Systems*, 34:22667–22681, 2021.
- Fout, A., Byrd, J., Shariat, B., and Ben-Hur, A. Protein interface prediction using graph convolutional networks. *Advances in neural information processing systems*, 30, 2017.
- Franceschi, L., Niepert, M., Pontil, M., and He, X. Learning discrete structures for graph neural networks. In *International conference on machine learning*, pp. 1972–1982. PMLR, 2019.
- Gneiting, T. and Raftery, A. E. Strictly Proper Scoring Rules, Prediction, and Estimation. *Journal of the American Statistical Association*, 102(477):359–378, 2007. ISSN 0162-1459.
- Goodfellow, I., Bengio, Y., Courville, A., and Bengio, Y. *Deep learning*, volume 1. MIT press Cambridge, 2016.
- Grathwohl, W., Choi, D., Wu, Y., Roeder, G., and Duvenaud, D. Backpropagation through the void: Optimizing control variates for black-box gradient estimation. In *International Conference on Learning Representations*, 2018.
- Grosse, R. and Martens, J. A kronecker-factored approximate fisher matrix for convolution layers. In *International Conference on Machine Learning*, pp. 573–582. PMLR, 2016.
- Gu, S., Levine, S., Sutskever, I., and Mnih, A. Muprop: Unbiased backpropagation for stochastic neural networks. In *4th International Conference on Learning Representations (ICLR)*, May 2016.
- Huang, S., Dossa, R. F. J., Raffin, A., Kanervisto, A., and Wang, W. The 37 implementation details of proximal policy optimization. In *ICLR Blog Track*, 2022. URL <https://iclr-blog-track.github.io/2022/03/25/ppo-implementation-details/>.

- Huijben, I. A., Kool, W., Paulus, M. B., and Van Sloun, R. J. A review of the gumbel-max trick and its extensions for discrete stochasticity in machine learning. *IEEE transactions on pattern analysis and machine intelligence*, 45(2): 1353–1371, 2022.
- Jacobs, R. A., Jordan, M. I., Nowlan, S. J., and Hinton, G. E. Adaptive mixtures of local experts. *Neural computation*, 3(1):79–87, 1991.
- Jang, E., Gu, S., and Poole, B. Categorical reparameterization with gumbel-softmax. In *International Conference on Learning Representations*, 2017.
- Jaynes, E. T. Information theory and statistical mechanics. *Physical review*, 106(4):620, 1957.
- Jeffares, A. and Liu, L. An introduction to discrete variational autoencoders. *arXiv preprint arXiv:2505.10344*, 2025.
- Kingma, D. P. and Ba, J. Adam: a method for stochastic optimization. In *International Conference on Learning Representations*, 2015.
- Kingma, D. P. and Welling, M. Auto-Encoding Variational Bayes. In *2nd International Conference on Learning Representations*, 2014.
- Kipf, T., Fetaya, E., Wang, K.-C., Welling, M., and Zemel, R. Neural relational inference for interacting systems. In *International conference on machine learning*, pp. 2688–2697. PMLR, 2018.
- Kipf, T. N. and Welling, M. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017.
- Kool, W., van Hoof, H., and Welling, M. Buy 4 reinforce samples, get a baseline for free! In *DeepRLStructPred@ICLR*, 2019. URL <https://api.semanticscholar.org/CorpusID:198489118>.
- Kool, W., van Hoof, H., and Welling, M. Estimating gradients for discrete random variables by sampling without replacement. In *International Conference on Learning Representations*, 2020.
- LeCun, Y. and Cortes, C. MNIST handwritten digit database. 2010. URL <http://yann.lecun.com/exdb/mnist/>.
- Liu, R., Regier, J., Tripuraneni, N., Jordan, M., and McAuliffe, J. Rao-blackwellized stochastic gradients for discrete distributions. In *International Conference on Machine Learning*, pp. 4023–4031. PMLR, 2019.
- Maddison, C., Mnih, A., and Teh, Y. The concrete distribution: A continuous relaxation of discrete random variables. In *Proceedings of the international conference on learning Representations*. International Conference on Learning Representations, 2017.
- Manenti, A., Zambon, D., and Alippi, C. Learning latent graph structures and their uncertainty. In *Forty-second International Conference on Machine Learning*, 2025. URL <https://openreview.net/forum?id=TMRh3ScSCb>.
- Martens, J. New insights and perspectives on the natural gradient method. *Journal of Machine Learning Research*, 21(146):1–76, 2020.
- Martins, A. and Astudillo, R. From softmax to sparsemax: A sparse model of attention and multi-label classification. In *International conference on machine learning*, pp. 1614–1623. PMLR, 2016.
- Matheson, J. E. and Winkler, R. L. Scoring rules for continuous probability distributions. *Management science*, 22(10):1087–1096, 1976.
- Miao, Y., Grefenstette, E., and Blunsom, P. Discovering discrete latent topics with neural variational inference. In *International conference on machine learning*, pp. 2410–2419. PMLR, 2017.
- Minervini, P., Franceschi, L., and Niepert, M. Adaptive perturbation-based gradient estimation for discrete latent variable models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pp. 9200–9208, 2023.
- Mnih, A. and Gregor, K. Neural variational inference and learning in belief networks. In *International Conference on Machine Learning*, pp. 1791–1799. PMLR, 2014.
- Mnih, A. and Hinton, G. E. A scalable hierarchical distributed language model. *Advances in neural information processing systems*, 21, 2008.
- Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., and Riedmiller, M. Playing atari with deep reinforcement learning. *arXiv preprint arXiv:1312.5602*, 2013.
- Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. Human-level control through deep reinforcement learning. *nature*, 518(7540): 529–533, 2015.
- Mohamed, S., Rosca, M., Figurnov, M., and Mnih, A. Monte carlo gradient estimation in machine learning. *The Journal of Machine Learning Research*, 21(1):5183–5244, 2020.

- Mood, A., Graybill, F., and Boes, D. Introduction to the theory of statistics. 1974.
- Morin, F. and Bengio, Y. Hierarchical probabilistic neural network language model. In *International workshop on artificial intelligence and statistics*, pp. 246–252. PMLR, 2005.
- Nelder, J. A. and Wedderburn, R. W. Generalized linear models. *Journal of the Royal Statistical Society Series A: Statistics in Society*, 135(3):370–384, 1972.
- Niepert, M., Minervini, P., and Franceschi, L. Implicit mle: backpropagating through discrete exponential family distributions. *Advances in Neural Information Processing Systems*, 34:14567–14579, 2021.
- Pascanu, R. and Bengio, Y. Revisiting natural gradient for deep networks. *arXiv preprint arXiv:1301.3584*, 2013.
- Paszke, A., Gross, S., Chintala, S., Chanan, G., Yang, E., DeVito, Z., Lin, Z., Desmaison, A., Antiga, L., and Lerer, A. Automatic differentiation in pytorch. In *NIPS-W*, 2017.
- Paulus, M., Choi, D., Tarlow, D., Krause, A., and Maddison, C. J. Gradient estimation with stochastic softmax tricks. *Advances in Neural Information Processing Systems*, 33: 5691–5704, 2020.
- Peters, J. and Schaal, S. Policy gradient methods for robotics. In *2006 IEEE/RSJ international conference on intelligent robots and systems*, pp. 2219–2225. IEEE, 2006.
- Pflug, G. C. *Optimization of Stochastic Models: The Interface between Simulation and Optimization*. Springer Science & Business Media, 1996.
- Rao, C. R. et al. Information and the accuracy attainable in the estimation of statistical parameters. *Bull. Calcutta Math. Soc.*, 37(3):81–91, 1945.
- Ross, S. M. *Simulation*. Academic Press, 2006.
- Scarselli, F., Gori, M., Tsoi, A. C., Hagenbuchner, M., and Monfardini, G. The graph neural network model. *IEEE transactions on neural networks*, 20(1):61–80, 2008.
- Schulman, J., Moritz, P., Levine, S., Jordan, M., and Abbeel, P. High-dimensional continuous control using generalized advantage estimation. *arXiv preprint arXiv:1506.02438*, 2015.
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., and Klimov, O. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Shlomi, J., Battaglia, P., and Vlimant, J.-R. Graph neural networks in particle physics. *Machine Learning: Science and Technology*, 2(2):021001, 2020.
- Sutton, R. S., Barto, A. G., et al. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.
- Titsias, M. K. and Lázaro-Gredilla, M. Local expectation gradients for black box variational inference. *Advances in neural information processing systems*, 28, 2015.
- Towers, M., Kwiatkowski, A., Terry, J. K., Balis, J. U., De Cola, G., Deleu, T., Goulão, M., Kallinteris, A., Krimmel, M., KG, A., et al. Gymnasium: A standard interface for reinforcement learning environments. *CoRR*, 2024.
- Tucker, G., Mnih, A., Maddison, C. J., Lawson, J., and Sohl-Dickstein, J. Rebar: Low-variance, unbiased gradient estimates for discrete latent variable models. *Advances in Neural Information Processing Systems*, 30, 2017.
- Van Den Oord, A., Vinyals, O., et al. Neural discrete representation learning. *Advances in neural information processing systems*, 30, 2017.
- Williams, R. J. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine learning*, 8:229–256, 1992.
- Zambon, D., Cini, A., Livi, L., and Alippi, C. Graph state-space models. *arXiv preprint arXiv:2301.01741*, 2023.
- Zeiler, M. D., Krishnan, D., Taylor, G. W., and Fergus, R. Deconvolutional networks. In *2010 IEEE Computer Society Conference on computer vision and pattern recognition*, pp. 2528–2535. IEEE, 2010.

A. Intuition on Natural Gradient

This section provides additional intuition for why information geometry plays a central role in optimization and why a diagonal Fisher Information Matrix is desirable.

Gradient-based optimization implicitly relies on a geometric assumption about the parameter space. Given a function $\mathcal{L}(\theta)$, consider the following *local optimization problem*:

$$\theta_{t+1} = \arg \min_{\theta} \left\{ \underbrace{\mathcal{L}(\theta_t) + \nabla \mathcal{L}(\theta_t)^\top (\theta - \theta_t)}_{\text{Loss linearization}} + \underbrace{\frac{1}{2} \|\theta - \theta_t\|_2^2}_{\text{Regularization}} \right\}. \quad (16)$$

The solution to this problem is the standard gradient descent update:

$$\theta_{t+1} = \theta_t - \nabla \mathcal{L}(\theta_t).$$

Problem in (16) considers minimizing the linearization of $\mathcal{L}(\theta)$ around θ_t . Since the linearization around θ_t approximates $\mathcal{L}$ well only *close* to θ_t , a regularization term is added to penalize values of θ that are *far* from the linearization point.

Note that the notions of “locality”, “closeness”, and “distance” are enforced by the regularization term and, more specifically, by the Euclidean norm $\|\cdot\|_2^2$. Thus, gradient descent is the solution to a local optimization problem in which *parameter distance is measured using the Euclidean distance*.

More generally, locality could be measured with respect to an *arbitrary metric* D instead of the Euclidean norm $\|\cdot\|_2^2$. In the case of statistical models, D could be, for example, the Kullback–Leibler divergence between the parameterized distributions.

When the space is not Euclidean, the local geometry of the parameter space is characterized by a *Riemannian metric tensor* $G(\theta)$, which captures the local curvature of the space.

To account for this curvature, the gradient can and should be appropriately *modified*. This leads to the *natural gradient*, defined as

$$\tilde{\nabla} \mathcal{L}(\theta_t) = G^{-1}(\theta_t) \nabla \mathcal{L}(\theta_t). \quad (17)$$

The resulting update corresponds to the solution of the local minimization problem induced by the chosen metric D .

In the case of statistical models, when D is the Kullback–Leibler divergence, the metric tensor G coincides with the **Fisher Information Matrix** (FIM).

Consequently, **the presence of curvature causes the direction of steepest descent to deviate from the ordinary gradient. If G is diagonal, the majority of the distortions induced by the geometry are eliminated**, and the natural gradient reduces to a coordinate-wise rescaling of the standard gradient. In this regime, adaptive first-order methods such as Adam can approximate this rescaling through per-parameter normalization of gradient magnitudes.

For further intuition regarding the role of information geometry and natural gradients in optimization, we refer the reader to the extensive literature on the subject (Amari, 1998; Amari & Douglas, 1998; Amari & Nagaoka, 2000; Pascanu & Bengio, 2013; Amari, 2016; Amari et al., 2019; Martens, 2020) and the references therein.

B. Proof of Proposition 4.1

Here we prove that the Fisher Information Matrix for a categorical distribution parameterized by a *softmax* (i.e., $p_i = \frac{e^{s_i}}{\sum_{k=1}^K e^{s_k}}$) is:

$$G_{\text{softmax}}(s) = \begin{bmatrix} p_1(1-p_1) & -p_1p_2 & \cdots & -p_1p_K \\ -p_2p_1 & p_2(1-p_2) & \cdots & -p_2p_K \\ \vdots & \vdots & \ddots & \vdots \\ -p_Kp_1 & -p_Kp_2 & \cdots & p_K(1-p_K) \end{bmatrix}$$

Proof. For a single observation, let $C = (C_1, \dots, C_K)$ be a one-hot encoded vector with $C_{\bar{k}} = 1$ if the observed category is $\bar{k}$ and zero elsewhere. The log-likelihood is:

$$\log(p(C|s)) = \sum_{k=1}^K C_k \log(p_k) = \sum_{k=1}^K C_k \left(s_k - \log \left(\sum_{k'=1}^K e^{s_{k'}} \right) \right) \quad (18)$$

We have:

$$\frac{\partial \log(p(C_k = 1|s))}{\partial s_i} = \delta_{ki} - p_i \quad (19)$$

With δ_{ki} being the Kronecker delta.

The Fisher Information Matrix is:

$$G_{softmax}(s)_{ij} = \mathbb{E}_{C \sim p(C|s)} \left[\frac{\partial \log(p(C|s))}{\partial s_i} \frac{\partial \log(p(C|s))}{\partial s_j} \right] \quad (20)$$

$$= \sum_{k=1}^K p_k (\delta_{ki} - p_i) (\delta_{kj} - p_j) \quad (21)$$

- For diagonal elements:

$$G_{softmax}(s)_{ii} = \sum_{k=1}^K p_k (\delta_{ki} - p_i)^2 = p_i (1 - p_i) \quad (22)$$

- For off-diagonal elements:

$$G_{softmax}(s)_{ij} = \sum_{k=1}^K p_k (\delta_{ki} - p_i) (\delta_{kj} - p_j) \quad (23)$$

$$= \sum_{k=1}^K p_k (\delta_{ki} \delta_{kj} - \delta_{ki} p_j - \delta_{kj} p_i + p_i p_j) \quad (24)$$

$$= -p_i p_j \quad (25)$$

yielding the Fisher Information Matrix $G_{softmax}(s)$ stated in the proposition. $\square$

C. Proof of Theorem 4.2

To prove Theorem 4.2, we use the following series of lemmas and propositions.

Lemma C.1. Given a set of bits $[b_1, \dots, b_H]$ then $a_{\boxed{ID}}$ is an ancestor of $p_{b_1, \dots, b_H}$ if and only if $\boxed{ID} = \boxed{b_1, \dots, b_{h-1}}$

Proof. By construction the binary number $[b_1, \dots, b_H]$ in $p_{b_1, \dots, b_H}$ represents the binary decisions taken at each hierarchy level h . In particular, the first $h-1$ terms $[b_1, \dots, b_{h-1}]$ represent, in order, the first $h-1$ binary decisions.

For each hierarchy level h , each node $a_{\boxed{ID}}$ is identified by $\boxed{ID}$. The numerical value represents its position reading right to left by construction (i.e., $b_h = 1$ corresponds to descending left). The first bit in $\boxed{ID}$ splits the 2^{h-1} numbers in half (i.e., for the left half the first bit is one, for the right half is zero). The subsequent bits recursively split the selected group in half following the same logic. Thus, each bit in $\boxed{ID}$ can be viewed as a binary decision of moving left or right. Thus, $a_{\boxed{b_1, \dots, b_{h-1}}}$ is the node reached from the root following binary decisions $b_1, \dots, b_{h-1}$. Since each p has H ancestors and the root is common the lemma is proved. $\square$

Lemma C.2. Given a_α and p_γ ,

$$\text{if } p_\gamma \text{ is not a descendant of } a_\alpha \implies \frac{\partial}{\partial a_\alpha} \log(p_\gamma) = 0. \quad (26)$$

Proof. Consider the binary representation of a_α . From Lemma C.1 $a_{\boxed{\text{ID}}}$ is not an ancestor of $p_{b_1, \dots, b_H}$ then $\boxed{\text{ID}} \neq [b_1, \dots, b_{h-1}]$. In that case $a_{\boxed{\text{ID}}}$ is not a term in (8) and thus $\frac{\partial}{\partial a_\alpha} \log(p_\gamma) = \frac{1}{p_\gamma} \frac{\partial}{\partial a_\alpha} p_\gamma = 0$. $\square$

Corollary C.3. Given a_α , a_β and p_γ with $\alpha \neq \beta$,

$$\text{If } a_\alpha \text{ is neither a descendant nor an ancestor of } a_\beta \implies \frac{\partial}{\partial a_\alpha} \log(p_\gamma) \frac{\partial}{\partial a_\beta} \log(p_\gamma) = 0.$$

Proof. If a_α is neither a descendant nor an ancestor of a_β then they do not share any descendant and thus by Lemma C.2 the Corollary is trivially proved. $\square$

Proposition C.4. The Fisher Information Matrix $G_a(s)$ for the catnat parameterization is diagonal.

Proof. To prove the Proposition we prove that all the off-diagonal terms of $G_a(s)$ are zero. By definition:

$$\begin{aligned} G_a(s)_{\alpha\beta} &= \mathbb{E}_{C \sim p(C|s)} \left[\frac{\partial}{\partial s_\alpha} \log(p(C|s)) \frac{\partial}{\partial s_\beta} \log(p(C|s)) \right] \\ &= \mathbb{E}_{C \sim p(C|s)} \left[\frac{\partial a_\alpha}{\partial s_\alpha} \frac{\partial}{\partial a_\alpha} \log(p(C|s)) \frac{\partial a_\beta}{\partial s_\beta} \frac{\partial}{\partial a_\beta} \log(p(C|s)) \right] \\ &= \sum_{\vec{b} \in \{0,1\}^H} p_{\vec{b}} \left[\frac{\partial a_\alpha}{\partial s_\alpha} \frac{\partial}{\partial a_\alpha} \log(p_{\vec{b}}) \frac{\partial a_\beta}{\partial s_\beta} \frac{\partial}{\partial a_\beta} \log(p_{\vec{b}}) \right] \end{aligned} \quad (27)$$

From Corollary C.3 if a_α is neither a descendant nor an ancestor of a_β the term in the square brackets is zero. We thus consider a_α being an ancestor of a_β , since the FIM is symmetric this is not restrictive. From Lemma C.2 the only terms that may produce nonzero addends are from the $\vec{b}$ that are descendant $\mathcal{D}_\beta$ of a_β . Thus:

$$G_a(s)_{\alpha\beta} = \sum_{\vec{b} \in \mathcal{D}_\beta} p_{\vec{b}} \left[\frac{\partial a_\alpha}{\partial s_\alpha} \frac{\partial}{\partial a_\alpha} \log(p_{\vec{b}}) \frac{\partial a_\beta}{\partial s_\beta} \frac{\partial}{\partial a_\beta} \log(p_{\vec{b}}) \right] \quad (28)$$

We call h_α and h_β the hierarchies of a_α and a_β and consider their binary representation $a_{\boxed{\text{ID}}_\alpha}$ and $a_{\boxed{\text{ID}}_\beta}$. From Equation 8 we write the log-likelihood derivative as:

$$\frac{\partial}{\partial a_{\boxed{\text{ID}}}} \log(p_{b_1, \dots, b_H}) = \mathbb{1} [[b_1, \dots, b_{h-1}] = \boxed{\text{ID}}] \frac{2b_h - 1}{a_{\boxed{\text{ID}}}^{b_h} (1 - a_{\boxed{\text{ID}}})^{(1-b_h)}} \quad (29)$$

where $\mathbb{1}[\cdot]$ is the indicator function, which evaluates to 1 if the given condition is true and 0 if it is false. The $\vec{b} \in \mathcal{D}_\beta$ share the same first $h_\beta - 1$ bits. Since a_α is an ancestor of a_β then $h_\beta > h_\alpha$ and thus b_{h_α} is the same for all $\vec{b} \in \mathcal{D}_\beta$. Then:

$$\begin{aligned} G_a(s)_{\alpha\beta} &= K \sum_{\vec{b} \in \mathcal{D}_\beta} p_{\vec{b}} \left[\frac{\partial}{\partial a_\beta} \log(p_{\vec{b}}) \right] \\ &= K \sum_{\substack{\vec{b} \in \mathcal{D}_\beta \\ b_{h_\beta}=1}} p_{\vec{b}} \underbrace{\left[\frac{\partial}{\partial a_\beta} \log(p_{\vec{b}}) \right]}_{a_{\boxed{\text{ID}}_\beta}^{-1}} + K \sum_{\substack{\vec{b} \in \mathcal{D}_\beta \\ b_{h_\beta}=0}} p_{\vec{b}} \underbrace{\left[\frac{\partial}{\partial a_\beta} \log(p_{\vec{b}}) \right]}_{-(1-a_{\boxed{\text{ID}}_\beta})^{-1}} \\ &= K \sum_{\substack{\vec{b} \in \mathcal{D}_\beta \\ b_{h_\beta}=1}} \frac{p_{\vec{b}}}{a_{\boxed{\text{ID}}_\beta}} - K \sum_{\substack{\vec{b} \in \mathcal{D}_\beta \\ b_{h_\beta}=0}} \frac{p_{\vec{b}}}{1 - a_{\boxed{\text{ID}}_\beta}} \\ &= \frac{K}{a_{\boxed{\text{ID}}_\beta}} \sum_{\substack{\vec{b} \in \mathcal{D}_\beta \\ b_{h_\beta}=1}} p_{\vec{b}} - \frac{K}{1 - a_{\boxed{\text{ID}}_\beta}} \sum_{\substack{\vec{b} \in \mathcal{D}_\beta \\ b_{h_\beta}=0}} p_{\vec{b}} \end{aligned} \quad (30)$$

By construction:

$$\begin{aligned} \sum_{\substack{\vec{b} \in \mathcal{D}_\beta \\ b_{h_\beta}=1}} p_{\vec{b}} &= P\left(\text{descend to node } a_{[\text{ID}]_\beta}\right) \cdot a_{[\text{ID}]_\beta} \\ \sum_{\substack{\vec{b} \in \mathcal{D}_\beta \\ b_{h_\beta}=0}} p_{\vec{b}} &= P\left(\text{descend to node } a_{[\text{ID}]_\beta}\right) \cdot (1 - a_{[\text{ID}]_\beta}) \end{aligned}$$

Thus, $G_a(s)_{\alpha\beta} = 0$ for off-diagonal terms. $\square$

Proposition C.5. *The diagonal terms of the Fisher Information Matrix $G_a(s)$ for the catnat parameterization are:*

$$G_a(s)_{ii} = P(a_i) \left(\frac{\partial a_i}{\partial s_i} \right)^2 \left(\frac{1}{a_i(1-a_i)} \right) \quad (31)$$

Proof. Reusing arguments from the previous proofs we can write:

$$\begin{aligned} G_a(s)_{ii} &= \mathbb{E}_{C \sim p(C|s)} \left[\left(\frac{\partial}{\partial s_i} \log(p(C|s)) \right)^2 \right] \\ &= \sum_{\vec{b} \in \mathcal{D}_i} p_{\vec{b}} \left(\frac{\partial}{\partial s_i} \log(p_{\vec{b}}) \right)^2 \\ &= \sum_{\vec{b} \in \mathcal{D}_i} p_{\vec{b}} \left(\frac{\partial a_i}{\partial s_i} \frac{\partial}{\partial a_i} \log(p_{\vec{b}}) \right)^2 \\ &= \left(\frac{\partial a_i}{\partial s_i} \right)^2 \sum_{\vec{b} \in \mathcal{D}_i} p_{\vec{b}} \left(\frac{\partial}{\partial a_i} \log(p_{\vec{b}}) \right)^2 \\ &= \left(\frac{\partial a_i}{\partial s_i} \right)^2 \left[\sum_{\substack{\vec{b} \in \mathcal{D}_i \\ b_{h_i}=1}} p_{\vec{b}} \left(\frac{1}{a_{[\text{ID}]_i}} \right)^2 + \sum_{\substack{\vec{b} \in \mathcal{D}_i \\ b_{h_i}=0}} p_{\vec{b}} \left(\frac{-1}{1 - a_{[\text{ID}]_i}} \right)^2 \right] \\ &= \left(\frac{\partial a_i}{\partial s_i} \right)^2 \left[P(a_i) \left(\frac{1}{a_i} \right)^2 \cancel{\not{a_i}} + P(a_i) \left(\frac{1}{1-a_i} \right)^2 \cancel{(1-a_i)} \right] \\ &= P(a_i) \left(\frac{\partial a_i}{\partial s_i} \right)^2 \left(\frac{1}{a_i(1-a_i)} \right) \end{aligned}$$

$\square$

Theorem 4.2 follows naturally from Proposition C.4 and Proposition C.5.

D. Proof of Corollary 4.3

Proof. To prove the corollary we start from Theorem 4.2 and substitute the definition in (12).

For the natural activation function ν :

$$\begin{aligned}
 \left(\frac{\partial \nu_i}{\partial s_i}\right)^2 \left(\frac{1}{\nu_i(1-\nu_i)}\right) &= \left(\frac{\partial}{\partial s_i} \frac{1 + \sin\left(\frac{\pi(s_i - C)}{A}\right)}{2}\right)^2 \left(\frac{1}{\left(\frac{1 + \sin\left(\frac{\pi(s_i - C)}{A}\right)}{2}\right) \left(1 - \frac{1 + \sin\left(\frac{\pi(s_i - C)}{A}\right)}{2}\right)}\right) \\
 &= \left(\frac{\pi}{2A} \cos\left(\frac{\pi(s_i - C)}{A}\right)\right)^2 \left(\frac{4}{\left(1 + \sin\left(\frac{\pi(s_i - C)}{A}\right)\right) \left(1 - \sin\left(\frac{\pi(s_i - C)}{A}\right)\right)}\right) \\
 &= \left(\frac{\pi}{2A} \cos\left(\frac{\pi(s_i - C)}{A}\right)\right)^2 \left(\frac{4}{1 - \sin^2\left(\frac{\pi(s_i - C)}{A}\right)}\right) \\
 &= \left(\frac{\pi}{2A} \cos\left(\frac{\pi(s_i - C)}{A}\right)\right)^2 \left(\frac{4}{\cos^2\left(\frac{\pi(s_i - C)}{A}\right)}\right) \\
 &= \frac{\pi^2}{A^2}
 \end{aligned}$$

Thus,

$$G_\nu(s)_{ii} = P(a_i) \frac{\pi^2}{A^2} \quad (32)$$

□

E. Experimental Details: Graph Structure Learning

This appendix summarizes the experimental setup for the Graph Structure Learning experiment, adapted from (Manenti et al., 2025).

E.1. Data-Generating Process

The dataset is generated from a system model comprising two components: a latent graph distribution $P_A^{\theta^*}$ that produces a random adjacency matrix A , and a Graph Neural Network f_{ψ^*} that maps an input feature matrix x and the graph A to an output y .

LATENT GRAPH DISTRIBUTION

The latent graph structure A is sampled from a multivariate Bernoulli distribution parameterized by a matrix of probabilities θ_{ij}^* :

$$P_{\theta^*}(A) = \prod_{i,j} (\theta_{ij}^*)^{A_{ij}} (1 - \theta_{ij}^*)^{1-A_{ij}} \quad (33)$$

Each entry A_{ij} represents a potential edge, sampled independently with a success probability of θ_{ij}^* . The ground-truth parameters θ_{ij}^* are set to the same non-zero value θ^* for edges forming the community structure depicted in Figure 5 and are zero otherwise. For the experiments we use a graph with 4 communities.

GNN ARCHITECTURE

The GNN function f_{ψ^*} used to process the sampled graph A and a random input feature matrix $x \in \mathbb{R}^{N \times d_{in}}$ is a GCN (Kipf & Welling, 2017). The input features are sampled from a normal distribution, $x \sim \mathcal{N}(0, \sigma_x^2 \mathbb{I})$ with $\sigma_x = 1$.

This generation process yielded a dataset of 10,000 input-output (x, y) pairs, partitioned into a training set (80%), a validation set (10%), and a test set (10%). The learnable model we train has an identical architecture to the one described above.

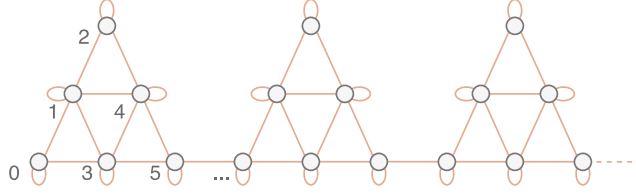


Figure 5. The base graph structure used to generate adjacency matrices for the experiments in Section 5.1. The matrices are sampled as subgraphs from this structure, where each orange edge is included with an independent probability of θ_{ij}^* , according to the distribution $P_{\theta^*}(A)$ in (33). Image taken from Manenti et al. (2025)

E.2. Learnable Model

For the learnable deep learning architecture, we employ the same class of latent graph distribution and GNN architecture as in the data-generating model. We jointly learn the parameters of the latent graph distribution, denoted by θ , and the parameters of the GNN, denoted by ψ .

We perform a two-stage grid search over learning rates. In all experiments, the learning rate for the GNN parameters ψ and the latent graph parameters θ is chosen from the same grid. The search procedure is:

1. Coarse grid: Test rates in $\{0.005, 0.01, 0.02, 0.05, 0.1, 0.2, 0.5\}$.
2. Refined grid: Centered around the best-performing coarse rate (selected by validation loss). The refined grids used in our experiments were:
 - Sigmoid parameterization: $\{0.025, 0.03, 0.037, 0.045, 0.055, 0.067, 0.082\}$.
 - Natural parameterization: $\{0.012, 0.015, 0.018, 0.022, 0.027, 0.033, 0.041\}$.

The final learning rate for each run is the one that yields the lowest validation loss. Using this best learning rate, we train 10 models to compute aggregate statistics.

E.3. Score Function Gradient Estimator

To compute gradients with respect to the parameters θ_{ij} of the latent graph distribution, we use the Score Function Gradient Estimator (SFGE), also known as REINFORCE (Williams, 1992; Mohamed et al., 2020). The SFGE allows us to estimate the gradient of an expectation of a function $\mathcal{L}(A)$ as follows:

$$\nabla_{\theta} \mathbb{E}_{A \sim P_{\theta}(A)} [\mathcal{L}(A)] = \mathbb{E}_{A \sim P_{\theta}(A)} [\mathcal{L}(A) \nabla_{\theta} \log P_{\theta}(A)]$$

A known issue with the SFGE is its high variance (Mohamed et al., 2020). To mitigate this, we incorporate a baseline term, which reduces variance without introducing bias into the gradient estimate. The gradient is then computed as:

$$\nabla_{\theta} \mathbb{E}_{A \sim P_{\theta}(A)} [(\mathcal{L}(A) - b) \nabla_{\theta} \log P_{\theta}(A)]$$

We use a multi-sample baseline where, for each sample in a batch of M sampled graphs, the baseline b is constructed using the estimate of the loss from the other $M - 1$ samples.

E.4. Loss Function

The model is trained to learn both the GNN parameters ψ and the graph distribution parameters θ by minimizing the Energy Score (ES) (Gneiting & Raftery, 2007). The ES is a multivariate extension of the Continuous Ranked Probability Score (CRPS), a proper scoring rule (Matheson & Winkler, 1976) that quantifies the compatibility between the model’s predictive distribution and the ground-truth observation y .

Given M adjacency matrices $\{A_m\}_{m=1}^M$ sampled from the latent graph distribution $P_{\theta}(A)$, the empirical ES loss is defined as:

$$\mathcal{L}_{\text{ES}} = \frac{1}{M} \sum_{m=1}^M \|f_{\psi}(x, A_m) - y\|_2 - \frac{1}{2M(M-1)} \sum_{m \neq n} \|f_{\psi}(x, A_m) - f_{\psi}(x, A_n)\|_2$$

E.5. Additional Training Parameters

All models are implemented in PyTorch (Paszke et al., 2017) and trained with the Adam optimizer (Kingma & Ba, 2015). We use a weight decay of 0, a batch size of 64, M equal to 32 and 40 epochs per run. Scores were initialized so that $\theta_{ij} \sim \mathcal{U}(0, 0.1)$.

F. Experimental Details: VAE

This appendix summarizes the setup for the experiments with Variational Autoencoders, which we adopt from the code available at the following link: <https://github.com/jxmorris12/categorical-vae>.

F.1. Model Architecture

The Variational Autoencoder (VAE) is composed of an encoder network $q_{\vec{s}}(\vec{C}|x)$ and a decoder network $f_{\psi}(\hat{y}|\vec{C})$. Both networks are implemented with a convolutional structure.

ENCODER & CATEGORICAL LATENT DISTRIBUTION

The encoder processes an input image $x \in \mathbb{R}^{28 \times 28}$, computes a tensor of scores $\vec{s} \in \mathbb{R}^{N \times K}$ – where N is the number of latent categorical variables, and K is the number of classes for each variable – and outputs the latent probabilities $q_{\vec{s}}(\vec{C}|x)$. The default convolutional architecture consists of 3 convolutional layers with ReLU activations, followed by 2 fully-connected layers.

Thus, the latent space is defined by N independent categorical random variables, with each variable taking one of K discrete states. The scores $\vec{s} \in \mathbb{R}^{N \times K}$ computed by the encoder are transformed into latent probabilities $q_{\vec{s}}(\vec{C}|x)$ with different parameterizations. We test three schemes for this parameterization: the *softmax* function, the *catnat* parameterization with *sigmoid* activation function and the *catnat* parameterization with *natural* activation function.

DECODER

The decoder takes a set of one-hot latent samples (one vector for each categorical distribution) $\vec{C}$ and processes it through 2 fully-connected layers and 3 transposed convolutional layers to reconstruct an image. A sigmoid activation function in the final layer ensures the output values are bounded within $[0, 1]$. Thus, the decoder $f_{\psi}(\hat{y}|\vec{C})$ produces a reconstruction whose entries can be interpreted as independent Bernoulli distributions over each pixel.

F.2. Hyperparameter search

We perform a two-stage grid search over learning rates. The search procedure is:

1. Coarse grid: Test rates in $\{0.0003, 0.001, 0.003, 0.01, 0.03\}$.
2. Refined grid: Centered around the best-performing coarse rate (selected by validation loss). The refined grid used for all parameterizations was $\{0.0045, 0.0056, 0.0069, 0.0085, 0.01, 0.013, 0.016, 0.02\}$.

The final learning rate for each run is the one that yields the lowest validation loss. Using this best learning rate, we train 5 models to compute aggregate statistics.

F.3. Gumbel-Softmax Reparameterization

To maintain a differentiable computation graph, we use the Gumbel-Softmax trick (Jang et al., 2017) to approximate sampling from $q_{\vec{s}}(\vec{C}|x)$. The temperature hyperparameter τ controls the smoothness of the approximation; as $\tau \rightarrow 0$, the samples converge to discrete one-hot vectors. During training, τ is annealed from an initial value of 1 to a minimum of 0.5 using an exponential decay rate of 3×10^{-5} . In the forward pass, we replace the dense $\vec{C}$ with its hard one-hot version while propagating gradients through the relaxed sample using the Straight-Through estimator (Bengio et al., 2013).

F.4. Loss Function

The model is trained by maximizing the Evidence Lower Bound (ELBO), which is bounded by the loss $\mathcal{L}_{\text{ELBO}} = \mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{KL}}$.

RECONSTRUCTION LOSS Given the decoder’s output, the reconstruction loss $\mathcal{L}_{\text{recon}}$ is the binary cross-entropy (BCE) between the input and the output, averaged over the batch:

$$\mathcal{L}_{\text{recon}} = -\frac{1}{B} \sum_{i=1}^B \mathbb{E}_{\vec{C} \sim q_{\vec{s}}(\vec{C}|x_i)} [\log f_{\psi}(x_i|\vec{C})]$$

KL DIVERGENCE The term $\mathcal{L}_{\text{KL}}$ is the Kullback-Leibler divergence between the approximate posterior $q_{\vec{s}}(\vec{C}|x)$ and a fixed prior $p(\vec{C})$. The prior is a set of N independent, uniform categorical distributions, i.e., $p(\vec{C}_n) = \text{Categorical}([\frac{1}{K}, \dots, \frac{1}{K}])$ for each latent variable n . The KL divergence is calculated analytically, summed over the N variables, and averaged over the batch:

$$\mathcal{L}_{\text{KL}} = \frac{1}{B} \sum_{i=1}^B D_{\text{KL}}(q_{\vec{s}}(\vec{C}_i|x_i)||p(\vec{C})) = \frac{1}{B} \sum_{i=1}^B \sum_{n=1}^N D_{\text{KL}}(q_{\vec{s}}(\vec{C}_{i,n}|x_i)||p(\vec{C}_n))$$

G. Additional Results: VAE with SGD

Here we report additional experiments in which the same categorical VAE models are trained using plain Stochastic Gradient Descent (SGD) instead of Adam. As shown in Table 5, the overall performance trends closely mirror those observed with Adam. Across all (N, K) configurations and for both MNIST and Binary MNIST, the proposed *catnat* parameterizations achieve lower test negative log-likelihoods than the *softmax* and *sparsemax* baselines. While absolute performance degrades for all methods under SGD, the relative advantage of *catnat* remains consistent. The two activation variants, σ and ν , again perform similarly, with small, configuration-dependent differences. These results support the claim that the benefits of the proposed parameterization are not optimizer-specific and extend to less adaptive, first-order optimization methods.

Table 5. Test set negative log likelihood on the MNIST dataset. Negative log-likelihoods are estimated with 512 importance samples (Burda et al., 2016). Models are compared across the number of categorical variables N , categories K , and categorical parameterizations. **Bold** numbers indicate the best-performing models (p-value of the Welch’s t-test < 0.05). Stochastic Gradient Descent used for optimization.

N	Param.	MNIST			Binary MNIST		
		$K = 8$	$K = 16$	$K = 32$	$K = 8$	$K = 16$	$K = 32$
10	<i>sparsemax</i>	102.9 $\pm$ 0.7	102.8 $\pm$ 0.4	107.4 $\pm$ 1.4	87.5 $\pm$ 0.4	86.3 $\pm$ 1.1	90.1 $\pm$ 1.3
	<i>softmax</i>	102.3 $\pm$ 0.4	101.5 $\pm$ 0.8	102.4 $\pm$ 0.5	86.7 $\pm$ 0.7	83.4 $\pm$ 0.3	84.9 $\pm$ 2.7
	<i>catnat</i> σ	100.6 $\pm$ 0.3	99.3 $\pm$ 0.9	100.1 $\pm$ 1.1	84.0 $\pm$ 0.6	80.1 $\pm$ 0.2	81.0 $\pm$ 0.5
	<i>catnat</i> ν	100.6 $\pm$ 0.4	99.6 $\pm$ 0.6	101.2 $\pm$ 1.4	83.8 $\pm$ 0.2	81.3 $\pm$ 0.4	80.9 $\pm$ 0.2
20	<i>sparsemax</i>	103.5 $\pm$ 1.6	112.9 $\pm$ 2.2	118.0 $\pm$ 2.7	85.3 $\pm$ 1.6	97.2 $\pm$ 1.9	107.7 $\pm$ 3.2
	<i>softmax</i>	99.3 $\pm$ 0.3	100.5 $\pm$ 1.1	106.1 $\pm$ 0.8	79.6 $\pm$ 0.5	80.7 $\pm$ 0.1	86.6 $\pm$ 2.9
	<i>catnat</i> σ	98.6 $\pm$ 0.3	99.7 $\pm$ 1.4	99.9 $\pm$ 0.5	78.7 $\pm$ 0.5	79.2 $\pm$ 0.7	81.5 $\pm$ 2.0
	<i>catnat</i> ν	98.8 $\pm$ 0.5	98.8 $\pm$ 0.4	100.7 $\pm$ 0.4	79.0 $\pm$ 0.9	79.2 $\pm$ 0.8	80.9 $\pm$ 0.9
30	<i>sparsemax</i>	105.7 $\pm$ 1.0	115.1 $\pm$ 2.0	123.7 $\pm$ 5.0	89.4 $\pm$ 0.6	101.7 $\pm$ 0.2	112.1 $\pm$ 2.0
	<i>softmax</i>	99.7 $\pm$ 0.4	100.6 $\pm$ 0.8	110.4 $\pm$ 5.7	80.1 $\pm$ 0.8	81.3 $\pm$ 0.5	85.1 $\pm$ 1.6
	<i>catnat</i> σ	99.8 $\pm$ 1.3	99.4 $\pm$ 0.5	104.8 $\pm$ 2.7	79.5 $\pm$ 1.0	79.3 $\pm$ 0.6	82.9 $\pm$ 1.4
	<i>catnat</i> ν	99.5 $\pm$ 0.2	100.2 $\pm$ 0.9	102.0 $\pm$ 1.0	79.7 $\pm$ 1.2	80.2 $\pm$ 1.9	83.5 $\pm$ 3.0

H. Additional Results: Different VAE Codebase

To further assess whether the empirical gains of the proposed parameterization are robust to implementation details, we performed an additional set of experiments using the discrete VAE codebase of Jeffares & Liu (2025).³ Starting from the original implementation, we made a small number of modifications to ensure convergence and to enable model selection. In particular, we trained for 150 epochs during the initial broad hyperparameter search and for 500 epochs in the final runs, since the original training schedule did not lead to full convergence in our setting. We also changed the batch size from 100 to 128 and split the original test set into separate validation and test subsets.

³The codebase is available at <https://github.com/alanjeffares/discreteVAE>.

Table 6. ELBO across the number of categorical variables N , categories K , and categorical parameterizations. **Bold** numbers indicate the best-performing model.

N	Param.	K		
		$K = 8$	$K = 16$	$K = 32$
10	<i>softmax</i>	-95.5 ± 0.4	-88.9 ± 0.3	-83.1 ± 0.2
	<i>catnat σ</i>	-86.2 ± 0.5	-79.5 ± 0.5	-73.1 ± 0.3
	<i>catnat ν</i>	-122.7 ± 33.4	-103.8 ± 16.6	-79.7 ± 8.2
20	<i>softmax</i>	-73.9 ± 0.4	-61.9 ± 0.4	-50.4 ± 0.3
	<i>catnat σ</i>	-66.7 ± 0.4	-53.5 ± 0.6	-41.8 ± 0.5
	<i>catnat ν</i>	-67.1 ± 0.5	-53.3 ± 0.7	-41.3 ± 0.5
30	<i>softmax</i>	-54.0 ± 0.3	-35.9 ± 0.1	-18.4 ± 0.4
	<i>catnat σ</i>	-47.2 ± 0.5	-28.0 ± 0.7	-10.1 ± 0.6
	<i>catnat ν</i>	-47.1 ± 0.7	-28.0 ± 0.7	-9.6 ± 0.4

For all parameterizations, we first searched over learning rates in $\{0.00005, 0.0001, 0.0002, 0.0005, 0.001, 0.002\}$. Based on validation performance, we then performed a finer search over values close to the best-performing learning rates from the first sweep, namely $\{0.0001, 0.00012, 0.00015, 0.00018, 0.00022, 0.00027, 0.00033, 0.0004\}$. The same tuning procedure was used for all parameterizations, and final test results were reported for the configuration with the best validation performance.

The resulting ELBO values are reported in Table 6. Consistent with the results obtained using our main VAE implementation, the best-performing model is always one of the *catnat* parameterizations. In particular, *catnat σ* performs best for smaller latent configurations, while *catnat ν* becomes competitive or best as the number of categorical variables increases. These results indicate that the advantage of the proposed categorical parameterization is not specific to a single VAE implementation, but persists across codebases and training pipelines.

I. Experimental Details: Reinforcement Learning

This appendix summarizes the experimental setup for the Reinforcement Learning experiments, which assess policy learning in discrete-action Atari environments. The implementation is adapted from the high-quality PPO implementation provided by (Huang et al., 2022).

I.1. Environment

We use the Breakout and Seaquest environments from the Atari Learning Environment (Bellemare et al., 2013), accessed via the Gymnasium library (Towers et al., 2024). The raw game frames undergo a standard preprocessing pipeline using a series of wrappers that, for example:

- Convert images to grayscale and resize them to 84×84 pixels.
- Stack 4 consecutive frames to capture temporal dynamics
- Clip the rewards to the range $[-1, 1]$ to stabilize training.

This setup is standard for benchmarking performance on Atari games (Mnih et al., 2015).

I.2. Model Architecture

The agent employs a shared-parameter actor-critic architecture with a convolutional network backbone:

- Shared Backbone: The network processes the stacked $4 \times 84 \times 84$ input observations, first normalizing pixel values by dividing by 255.0. It then passes through three convolutional layers with ReLU activations. The network architecture is:

1. 32 filters of size 8×8 with a stride of 4.
2. 64 filters of size 4×4 with a stride of 2.
3. 64 filters of size 3×3 with a stride of 1.

The output is flattened and passed through a fully-connected layer with 512 units (ReLU activated). All layers are initialized using orthogonal initialization.

- Policy and Value Heads: The 512-dimensional latent representation is fed into two separate linear heads:
 - The policy head (actor) outputs a vector of scores, one for each possible action.
 - The value head (critic) outputs a single scalar estimating the state-value.

We test two methods to convert the policy head’s scores into action probabilities: the standard *softmax* function and the *catmat* parameterization using the *natural* activation function.

I.3. Proximal Policy Optimization

The model is trained using the Proximal Policy Optimization (PPO) algorithm (Schulman et al., 2017). PPO is an on-policy algorithm that optimizes a clipped surrogate objective function. The total loss is a combination of the policy loss, the value function loss, and an entropy bonus to encourage exploration:

$$J(\theta) = \mathbb{E}_t [L_t^{\text{CLIP}}(\theta) - c_1 L_t^{\text{VF}}(\theta) + c_2 H[\pi_\theta](s_t)],$$

where the clipped surrogate is

$$L_t^{\text{CLIP}}(\theta) = \min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right),$$

Here, $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio, and $\hat{A}_t$ is the advantage estimate. Advantages are calculated using Generalized Advantage Estimation (GAE) (Schulman et al., 2015) with $\gamma = 0.99$ and $\lambda = 0.95$, and are normalized per mini-batch. The value loss is typically

$$L_t^{\text{VF}}(\theta) = \frac{1}{2} (V_\theta(s_t) - \hat{V}_t)^2,$$

and $H[\pi_\theta](s_t)$ denotes the policy entropy. The clipping ϵ and coefficients c_1, c_2 are hyperparameters.

I.4. Hyperparameter Optimization

Due to the high computational cost, we performed a targeted hyperparameter search instead of an exhaustive grid search. For each parameterization method and environment, we ran 160 trials using a Tree-structured Parzen Estimator (TPE) sampler (Bergstra et al., 2011) to find promising hyperparameter configurations. Table 7 summarizes each parameter’s type, sampling range, and any non-default scale or step size. The top 10 configurations identified by this search were then trained with 10 different random seeds to ensure more reliable performance statistics.

Table 7. Hyperparameter sweep: types, sampling ranges, and non-default scales/steps.

Parameter	Type	Range	Scale / Step / Notes
learning_rate	float	$5.0 \times 10^{-5} - 1.0 \times 10^{-2}$	log scale
num_steps	int	32 – 512	step = 32
update_epochs	int	1 – 16	step = 2
clip_coef	float	0.01 – 0.90	linear sampling
ent_coef	float	0.0 – 1.0	linear sampling
num_envs	int	8 – 16	step = 2
num_minibatches	int	2 – 16	step = 4
max_grad_norm	float	0.1 – 10.0	step = 0.1

I.5. Additional Training Parameters

We trained all models for a total of 8 million timesteps using the Adam optimizer (Kingma & Ba, 2015) with an ϵ of 10^{-5} . The learning rate, identified via hyperparameter search, was linearly annealed to zero over the course of training.